No	Nama Penyedia Layanan	Tautan Pendaftaran & Konsol Kunci (Eksisting & Gabungan)	Alternatif / Endpoint Spesifik & Status Audit 2026	Kapasitas Kuota Gratis, Batas Laju & Peran Rolling
				terdesentralisasi Bittensor.
2A.11	**Civitai & Spider Cloud API** (Model Unfiltered & Web Crawling API)	Civitai Account (Sumber 3): https://c[span_4](start_span)[span_4](end_span)ivitai.com/user/account	Spider Cloud API (Tanpa Kartu): https://spider.cloud	**100% Tanpa Kartu**. Kunci API Civitai digunakan untuk menarik *checkpoint* gambar tanpa filter dan *negative embeddings* secara terprogram, sedangkan Spider Cloud memberikan kunci API gratis tanpa kartu untuk *smart web scraping*.
2A.12	**Cerebras Cloud, GitHub Models & SambaNova** *(Status Transisi / Legacy)*	Cerebras: https://cloud.cerebras.ai (https://api.cerebras.ai/v1) GitHub Models: https://github.com/marketplace/models SambaNova: https://cloud.sambanova.ai	**Peringatan Otomasi Kode**: Akun lama Cerebras masih menikmati 1M token/hari (2.600+ tok/s pada WSE-3 untuk Llama 4 Scout, Qwen3 32B, gpt-oss-120b), namun per 16 Juli 2026 akun baru dialihkan ke uji coba $5 yang wajib kartu. GitHub Models pensiun pada 30 Juli 2026, dan SambaNova menghentikan kredit akun baru pada Agustus 2026.	Kode agen wajib membungkus ketiga penyedia ini di dalam blok try-except dengan *fallback* instan ke **Mistral (2A.2)**, **Google AI Studio (2A.1)**, dan **Groq (2A.3)** apabila menerima kode status HTTP 401, 402, 403, atau 404.

2B. Infrastruktur Hosting, Proksi Tepi, GPU Notebook, Memori & Database Gratis

(Tanpa Kartu Kredit)

Seluruh komponen di bawah ini berasal dari sub-kategori 2B (LINK SUMBER.pdf) serta layanan penyimpanan dan infrastruktur dari Kategori 1 dan Kategori 2 (Link_daftar_vps server.txt — khususnya **Backblaze B2** yang terbukti memberikan 10 GB pertama tanpa kartu Visa). Dengan menggabungkan Turso, Supabase, dan Neon, agen memiliki tiga mesin basis data SQL dan vektor terpisah yang saling mem-backup secara otomatis.

No	Nama Layanan Infrastruktur	Tautan Pendaftaran & Repositori (Eksisting & Gabungan)	Peran Utama & Rantai Rolling Failover Infrastruktur
2B.1	**Cloudflare Workers, Pages, R2 & Turnstile** (new-api-worker)	Skrip Proksi: https://github.com/Calcium-Ion/new-api-worker Dashboard: https://dash.cloudflare.com & https://dash.cloudflare.com/sign-up	**100% Tanpa Kartu**. Menyediakan 100.000 request/hari untuk menyembunyikan IP server agregator via *Transform Rules* (menghapus tajuk IP pengunjung), hosting gratis untuk *Mini App/Web Pages*, serta proteksi anti-bot Turnstile.
2B.2	**Turso Cloud, Supabase & Neon Serverless Postgres** (Rantai Failover Database SQL/Vektor)	Turso (5 GB Distributed SQLite/libSQL): https://turso.tech/ Supabase (500 MB Postgres + pgvector + 1 GB Storage): https://supabase.com Neon (500 MB Serverless Postgres): https://neon.tech	**100% Tanpa Kartu (Login via GitHub)**. Membentuk rantai *rolling* basis data: Turso menangani penulisan konkuren cepat (*concurrent writes*) untuk status transaksi agen; Supabase menyimpan memori vektor pgvector; dan Neon menyediakan SQL_DSN PostgreSQL terpusat untuk klaster one-api dan new-api-horizon.
2B.3	**Upstash Serverless Redis & Backblaze B2 Object Storage**	Upstash Redis: https://upstash.com Backblaze B2 (Sumber 3): https://www.backblaze.com/cloud-storage	**100% Tanpa Kartu**. Upstash menyediakan REDIS_CONN_STRING gratis untuk *caching* one-api dan antrean jhao104/proxy_pool. Backblaze B2 menyediakan **10 GB**

No	Nama Layanan Infrastruktur	Tautan Pendaftaran & Repositori (Eksisting & Gabungan)	Peran Utama & Rantai Rolling Failover Infrastruktur
			Object Storage gratis tanpa kartu Visa sebagai cadangan langsung (*failover*) untuk penyimpanan berkas media dan model di samping Supabase Storage dan Cloudflare R2.
2B.4	**Google Colaboratory & Kaggle Notebooks** (Komputasi GPU Cloud Gratis)	Google Colab (GPU T4): https://colab.research.google.com Kaggle (30 Jam/Minggu Dual-T4 / P100): https://www.kaggle.com	**100% Tanpa Kartu**. Digunakan secara bergantian (*rolling*) untuk mengeksekusi proses *abliteration* bobot model (jim-plus/llm-abliteration atau AIAnytime/ablate) maupun merender suara dan video ketika mesin lokal tidak memiliki GPU memadai.
2B.5	**Vercel, Mem0, Letta & E2B Sandbox** (Hosting, Memori Persisten & Eksekusi Kode Aman)	Vercel: https://vercel.com Mem0: https://github.com/mem0ai/mem0 Letta / MemGPT: https://github.com/letta-ai/letta-node E2B Code Sandbox: https://e2b.dev	**100% Tanpa Kartu (Login via GitHub)**. Mem0 (mem0 init --agent) dan Letta membentuk memori lintas sesi ganda (cloud dan lokal), Vercel menyajikan antarmuka web atau router ringan, dan E2B memberikan kredit kotak pasir gratis untuk mengeksekusi kode yang dihasilkan agen secara terisolasi.

2C. Identitas Pengembang, Bot Omnichannel & Gerbang Pembayaran Otomatis (100% Tanpa Kartu Visa)

Sub-kategori 2C mengintegrasikan komponen operasional dari Kategori 1 pada Link_daftar_vps server.txt yang memungkinkan agen beroperasi sebagai entitas bisnis otonom (*Autonomous Treasury*) yang menerima pembayaran lokal (QRIS) maupun global (Crypto USDT) tanpa memerlukan kartu perbankan Visa atau Mastercard. Dengan menyiapkan tiga token bot Telegram terpisah dan tiga gerbang pembayaran QRIS lokal yang didampingi kasir kripto

otomatis, sistem transaksi tetap berjalan meskipun salah satu penyedia gerbang pembayaran mengalami gangguan pemeliharaan.

No	Nama Layanan & Fungsi Operasional	Tautan Resmi (Dari Sumber Induk 3)	Mekanisme Integrasi & Failover Otomatis
2C.1	**GitHub Signup, Telegram @BotFather & @userinfobot**	GitHub Account: https://github.com/signup BotFather: https://t.me/BotFather UserInfoBot: https://t.me/userinfobot	Akun GitHub digunakan sebagai kunci OAuth 1-klik di seluruh layanan Kelompok 2. @BotFather menerbitkan 3 token independen (TOKEN_BOT_UTAMA, TOKEN_BOT_ACTIVITY, TOKEN_BOT_CADANGAN) sehingga jika bot utama terkena pembatasan Telegram, trafik otomatis beralih ke bot cadangan.
2C.2	**Meta for Developers** (WhatsApp & Messenger Resmi)	Portal: https://developers.facebook.com/	Login menggunakan akun Facebook untuk integrasi API resmi WhatsApp & Messenger; berpasangan dengan devlikeapro/waha (1E.4) sebagai jalur *failover* mandiri tanpa biaya per pesan.
2C.3	**Pakasir, Tripay & Duitku** (Rantai Failover Payment Gateway QRIS Indonesia)	Pakasir (Primer): https://pakasir.com/ Tripay (Sekunder): https://tripay.co.id/ Duitku (Tersier): https://www.duitku.com/	**100% Tanpa Kartu Visa**. Ketiga gerbang pembayaran QRIS lokal ini dirangkai secara *rolling*: apabila endpoint pembuatan faktur QRIS pada Pakasir mengalami pemeliharaan, kasir bot otomatis memanggil API Tripay atau Duitku sehingga arus kas masuk tidak pernah terhenti.
2C.4	**Telegram @CryptoBot** (Kasir Pembayaran	Bot Kasir: https://t.me/CryptoBot	**100% Tanpa Kartu Visa**. Diaktifkan hanya

No	Nama Layanan & Fungsi Operasional	Tautan Resmi (Dari Sumber Induk 3)	Mekanisme Integrasi & Failover Otomatis
	Crypto USDT Global)		dengan mengetik /start \rightarrow *Crypto Pay* \rightarrow *Create App* untuk mengambil CRYPTOBOT_API_TOKEN. Pendapatan USDT yang terkumpul dapat langsung dipakai untuk mengisi ulang saldo komputasi GPU di Novita AI, RunPod, Vast.ai, dan PiAPI tanpa pernah menyentuh rekening bank tradisional.

Kelompok 3: Wajib Kartu Visa/Mastercard ATAU Wajib Dompet Kripto (DePIN, Airdrop & Wholesale GPU)

Kelompok 3 menyatukan seluruh layanan dari **Kelompok 3 (LINK SUMBER.pdf)** dan **Kategori 2 (Link_daftar_vps server.txt)**, serta memperluas jaringan komputasi terdesentralisasi (DePIN) dengan ekosistem **Akash Network** (akash-network/node) dari hasil riset terbaru. Sesuai protokol keamanan finansial **FIN-SEC-01**, apabila arsitek sistem memutuskan untuk mengaktifkan layanan pada sub-kategori 3A, wajib diterbitkan satu Kartu Debit Visa Virtual terisolasi (misalnya melalui kantong terpisah bernama "Vault API bikinAI" pada aplikasi bank digital seperti Bank Jago atau Jenius dengan saldo terukur ~Rp 20.000 untuk verifikasi) dan dilarang keras menautkan kartu dari rekening tabungan utama.

3A. Penyedia Cloud, VPS Gratis Selamanya & API Grosir yang Membutuhkan Kartu Visa / Mastercard (atau Crypto USDT saat Top-Up)

Pada sub-kategori ini, sebagian besar platform API grosir (Fal.ai, Novita AI, RunPod, Vast.ai, Replicate, PiAPI, WaveSpeed AI) mengizinkan pembuatan akun gratis saat ini juga menggunakan login Google atau GitHub untuk mengambil kunci API dan menikmati bonus kredit pendaftaran awal (*Sign-up Free Credits* senilai $0,50 hingga $5+), sementara kartu Visa atau pembayaran Crypto USDT baru dibutuhkan ketika mengaktifkan fitur *Auto-Topup* dari pendapatan pelanggan.

No	Nama Layanan Cloud / API Grosir	Tautan Resmi (Gabungan Sumber 1 & Sumber 3)	Status Kartu Visa vs Crypto USDT & Peran dalam Sistem
3A.1	**Oracle Cloud Always Free Tier** (VPS ARM 24 Jam Gratis Selamanya)	Pendaftaran: https://www.oracle.com/cloud/free/	**Wajib Verifikasi Kartu Visa/Mastercard** (saldo minimal ~Rp 20.000 untuk pengecekan otorisasi yang langsung

No	Nama Layanan Cloud / API Grosir	Tautan Resmi (Gabungan Sumber 1 & Sumber 3)	Status Kartu Visa vs Crypto USDT & Peran dalam Sistem
			dikembalikan otomatis). Memberikan **4 ARM Ampere OCPUs + 24 GB RAM** gratis selamanya untuk menjalankan OmniRoute, 9Router, New-API Horizon, dan jhao104/proxy_pool 24/7.
3A.2	**Modal Serverless GPU, GCP, AWS & Fly.io**	Modal ($30/BulanKreditGPU$): h 300 Trial): https://cloud.google.com/free AWS Free Tier: https://aws.amazon.com/free/ Fly.io (Docker Hosting): https://fly.io	**Wajib Kartu Visa/Mastercard** untuk membuka kuota. Modal sangat bernilai strategis karena memberikan kredit komputasi GPU serverless sebesar **$30 setiap bulan secara berulang** setelah kartu diverifikasi, menjadikannya *fallback* utama untuk merender video Wan2.2 atau HunyuanVideo-1.5 di cloud dengan biaya Rp 0.
3A.3	**Novita AI, RunPod, Vast.ai & PiAPI** (Grosir GPU & Media — **Mendukung Crypto USDT Tanpa Kartu Visa**)	Novita AI: https://novita.ai/ RunPod (FlashBoot 24GB VRAM): https://www.runpod.io/ Vast.ai (Sewa RTX 3090/4090 Termurah): https://cloud.vast.ai/ PiAPI (Musik Suno v6 Studio & Udio): https://piapi.ai/	**Daftar Akun Gratis via Google/GitHub sekarang**. Keunggulan krusial keempat penyedia ini adalah dukungan pengisian saldo menggunakan **Crypto USDT** (misalnya dari pendapatan @CryptoBot pada 2C.4 via jaringan TRON, Solana, atau Coinbase Commerce) sehingga dapat dioperasikan sepenuhnya **tanpa kartu Visa**.
3A.4	**Fal.ai, WaveSpeed AI,**	Fal.ai (Kling 3.0, Veo,	**Daftar Akun Gratis via**

No	Nama Layanan Cloud / API Grosir	Tautan Resmi (Gabungan Sumber 1 & Sumber 3)	Status Kartu Visa vs Crypto USDT & Peran dalam Sistem
	Replicate & OpenAI Moderation (API Media & Moderasi Grosir)	Wan 2.2, Seedance, Flux): https://fal.ai/dashboard/keys WaveSpeed AI (Seedance 1.5 Pro & Wan 2.2): https://wavespeed.ai/ Replicate (4K Real-ESRGAN & Motion Control): https://replicate.com/account/api-tokens OpenAI Platform: https://platform.openai.com/api-keys	**GitHub/Google sekarang** untuk mengambil kunci API. Kartu Visa baru diperlukan saat mengaktifkan *Auto-Topup* komersial. Pada OpenAI, titik akhir omni-moderation-latest bersifat gratis untuk memfilter teks multibahasa, dan organisasi yang memenuhi syarat berbagi trafik dapat memperoleh hingga 1 juta token/hari pada model GPT-5.x serta 10 juta token/hari pada model kecil.

3B. Jaringan DePIN, Mining AI & Marketplace Komputasi Terdesentralisasi (Wajib Dompet Kripto & Spesifikasi GPU)

Proyek di dalam sub-kategori 3B tidak memerlukan kartu perbankan maupun akun Gmail, melainkan menggunakan autentikasi pasangan kunci kriptografi (dompet Bittensor coldkey/hotkey, dompet Solana, atau dompet Cosmos/Akash). Selain **Bittensor**, **Nosana**, dan **IO.net** dari berkas sumber, riset menambahkan **Akash Network** (akash-network/node, yang baru merilis versi v2.1.2 dan v3.0.0-a0 pada tahun 2026) sebagai pasar komputasi awan *peer-to-peer* yang dilengkapi keterampilan agen khusus (akash-skill) untuk men-deploy kontainer Docker secara otomatis.

No	Nama Protokol DePIN	Tautan Repositori & Dokumentasi (Eksisting & Ekspansi Riset)	Mekanisme Insentif, Rumus Skor & Peran dalam Ekosistem
3B.1	**Bittensor Ecosystem** (Subtensor Monorepo, Subnet 27 Compute & Score Predict)	Monorepo & SDK: https://github.com/RaoFoundation/bittensor (atau opentensor/bittensor) Subnet Template: https://github.com/opentensor/bittensor-subnet-template Testnet Guide: https://github.com/opentensor/bittensor-subnet-	Menggunakan dompet lokal (btcli wallet new_coldkey & new_hotkey). Menyediakan GPU menganggur (minimal 4–8 core CPU, 16 GB RAM, 24 GB VRAM A100, SSD 100 GB 5.000 IOPS) ke Subnet 27 atau menjalankan

No	Nama Protokol DePIN	Tautan Repositori & Dokumentasi (Eksisting & Ekspansi Riset)	Mekanisme Insentif, Rumus Skor & Peran dalam Ekosistem
		template/blob/main/docs/running_on_testnet.md Min Compute Spec: https://github.com/opentensor/bittensor-subnet-template/blob/main/min_compute.yml Subnet 27 Compute: https://github.com/commune-ai/compute-ni Score Predict Subnet: https://github.com/score-technologies/score-predict/blob/main/README.md	model *Random Forest* di score-predict menghasilkan imbalan token **TAO** melalui Yuma Consensus.
3B.2	**Nosana & IO.net** (Jaringan Klaster GPU Terdesentralisasi Berbasis Solana)	Nosana CLI: https://github.com/nosana-ci/nosana-cli Nosana LLM Benchmarking: https://github.com/nosana-ci/nosana-llm-benchmarking Nosana Portal: https://github.com/nosana-ci/nosana.io IO.net Launch Binaries: https://github.com/ionet-official/io_launch_binaries/issues/48	Wajib dompet kripto Solana (**SOL / NOS / IO**). Memungkinkan monetisasi GPU menganggur di luar jam kerja agen untuk menerima distribusi insentif atau airdrop, sekaligus menyediakan komputasi inferensi berbiaya jauh di bawah cloud tradisional.
3B.3	**Akash Network** *(Alternatif Superior Baru — Decentralized Serverless Cloud & Agent Deployment)*	Node Source Code: https://github.com/akash-network/node & Releases (v2.1.2 / v3.0.0-a0): https://github.com/ovrclk/akash/releases Install Script: https://raw.githubusercontent.com/akash-network/node/main/install.sh Provider Daemon: https://github.com/akash-network/provider Web Console (Deploy	Dibangun di atas Cosmos SDK dan CometBFT (berlisensi Apache-2.0) menggunakan token **AKT**. Berfungsi ganda: pemilik perangkat keras dapat menjalankan daemon provider untuk menyewakan kapasitas komputasi, sementara agen koding AI (seperti Claude Code atau OpenClaw) dapat menggunakan

No	Nama Protokol DePIN	Tautan Repositori & Dokumentasi (Eksisting & Ekspansi Riset)	Mekanisme Insentif, Rumus Skor & Peran dalam Ekosistem
		Docker): https://github.com/akash-network/console Agent SDL Generator Skill: https://github.com/akash-network/akash-skill Awesome Deployment Templates: https://github.com/akash-network/awesome-akash Helm Charts & RPC Proxy: https://github.com/akash-network/helm-charts & https://github.com/akash-network/rpc-proxy Docs Architecture: https://akash.network/docs/node-operators/architecture/	repositori akash-skill untuk menghasilkan berkas konfigurasi SDL secara otomatis dan men-deploy kontainer agregator ke jaringan Akash tanpa kartu kredit.

Mekanika Matematika Perutean Pintar, Abliteration Bebas Malware, dan Evaluasi SOTA

Keberhasilan operasional dari ratusan tautan di atas bergantung pada ketepatan matematis di tiga titik kendali: (1) algoritma perutean kueri yang menghemat kuota model frontier, (2) prosedur penghapusan sensor (*abliteration*) yang menjaga integritas kognitif model tanpa menyisipkan malware, dan (3) pemilihan model penalaran kompak yang mampu bersaing dengan model raksasa di perangkat keras lokal.

Formulasi Keputusan Smart Router dan Ekonomi DePIN

Ketika sebuah instruksi x masuk ke lapisan gateway, sistem terlebih dahulu memeriksa kecocokan semantik pada Valkey/RedisVL (*semantic cache* dengan latensi ~25 milidetik). Apabila terjadi *cache miss*, kueri diteruskan ke kerangka kerja **lm-sys/RouteLLM** yang memanfaatkan bobot faktorisasi matriks (routellm/mf_gpt4_augmented) hasil pelatihan pada data preferensi Chatbot Arena. Perute memproyeksikan representasi laten dari prompt x dan kapabilitas model untuk menghitung probabilitas kemenangan model kuat (\mathcal{M}_{\text{strong}}, seperti kuota gratis Gemini 3.5 Flash, Mistral Large, atau DeepSeek-R1 684B) terhadap model lemah atau lokal (\mathcal{M}_{\text{weak}}, seperti DeepSeek-R1-0528-Qwen3-8B atau Huihui-Qwen3.8-27B-abliterated di llama.cpp) menggunakan fungsi keputusan berambang batas \tau:

$$\text{Route}(x) = \begin{cases} \mathcal{M}_{\text{strong}} & \text{jika } P(\text{strong} \succ \text{weak} \mid x) \ge \tau \\ \mathcal{M}_{\text{weak}} & \text{jika } P(\text{strong} \succ \text{weak} \mid x) < \tau \end{cases}$$

Dengan menetapkan kalibrasi ambang batas standar $\tau = 0{,}11593$ pada perute *Matrix Factorization* (mf), sistem secara deterministik mengalokasikan tepat 50% kueri tersulit ke rantai API cloud dan 50% sisanya ke mesin inferensi lokal, memangkas konsumsi kuota token eksternal hingga 40%–70% tanpa penurunan kualitas yang terukur.

Apabila perangkat keras lokal memiliki siklus GPU yang menganggur saat beban kueri rendah, arsitektur ini dapat mengalihkan kapasitas GPU ke protokol **Bittensor Subnet 27** (commune-ai/compute-ni), di mana perolehan insentif token TAO ditentukan secara matematis oleh skor total node penambang (S_{total}):

$$S_{\text{GPU}} = \text{Perf}_{\text{base}} \times N_{\text{GPU}} \times \left(\frac{100}{\text{Max_Scenario}}\right), \quad S_{\text{total}} = S_{\text{GPU}} + B_{\text{alloc}}$$

Dalam persamaan tersebut, $\text{Perf}_{\text{base}}$ merepresentasikan kinerja dasar unit GPU, N_{GPU} adalah jumlah kartu grafis, Max_Scenario merupakan batas skenario tertinggi jaringan (contohnya $3{,}90 \times 8 = 31{,}2$ yang menghasilkan faktor pengali $\approx 3{,}2051$ untuk batas skor GPU 100), dan B_{alloc} adalah bonus alokasi sebesar 100 poin ketika mesin sedang dialokasikan secara aktif kepada validator melalui register.py (sehingga skor maksimum mencapai 200 poin). Pada subnet prediksi olahraga score-technologies/score-predict, imbalan penambang yang mengeksekusi pengklasifikasi *Random Forest* dikalikan lagi dengan *streak multiplier* (hingga $1{,}8\times$ untuk ≥ 20 prediksi benar berturut-turut) dan *7-day win rate multiplier* ($1{,}3\times$ untuk akurasi ≥ 70[span_19](start_span)[span_19](end_span)[span_21](start_span)[span_21](end_span)%, atau penalti $0{,}9\times$ jika di bawah 40\%).

Ortogonalisasi Ruang Residual (Abliteration) dan Perisai Serialisasi Bobot

Untuk melayani kueri penelitian akademik, pengujian penetrasi siber (*red-teaming*), dan instruksi tanpa filter tanpa memicu penolakan heuristik ("I'm sorry, I cannot fulfill this request"), arsitektur ini mengadopsi metodologi **Abliteration** yang berlandaskan pembuktian Arditi et al. (andyrdt/refusal_direction) bahwa perilaku penolakan pada transformator dimediasi oleh satu arah vektor tunggal di dalam aliran residual (*residual stream*). Dengan mengevaluasi himpunan prompt sensitif $\mathcal{D}_{\text{harmful}}$ (berukuran N) dan himpunan prompt netral $\mathcal{D}_{\text{harmless}}$ (berukuran M) pada posisi token instruksi terakhir di lapisan kandidat l, vektor perbedaan rata-rata $\mathbf{v}^{(l)}$ dan vektor satuan arah penolakan $\hat{\mathbf{r}}$ diekstraksi sebagai berikut:

$$\mathbf{v}^{(l)} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{a}_{\text{harmful}, i}^{(l)} - \frac{1}{M} \sum_{j=1}^{M} \mathbf{a}_{\text{harmless}, j}^{(l)}, \quad \hat{\mathbf{r}} = \frac{\mathbf{v}^{(l)}}{\Vert\mathbf{v}^{(l)}\Vert_2}$$

Agar penghapusan sensor tidak mengikis kemampuan penalaran umum atau memicu nilai NaN dan Inf pada model sensitif (sebagaimana sering terjadi pada *Simple Abliteration* generasi pertama), implementasi produksi pada jim-plus/llm-abliteration dan Orion-zhen/abliteration menerapkan **Biprojection** (*Gram-Schmidt orthogonalization*) terhadap arah rata-rata aktivasi tidak berbahaya yang telah dinormalisasi ($\hat{\mathbf{d}}_{\text{harmless}}$) sebelum memodifikasi matriks bobot $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$:

$\mathbf{r}_{\perp} = \hat{\mathbf{r}} - (\hat{\mathbf{r}} \cdot \hat{\mathbf{d}}_{\text{harmless}})\hat{\mathbf{d}}_{\text{harmless}}, \quad \hat{\mathbf{r}}_{\text{proj}} = \frac{\mathbf{r}_{\perp}}{\Vert\mathbf{r}_{\perp}\Vert_2}$
$\mathbf{W}_{\text{new}} = \left(\mathbf{I} - \alpha \hat{\mathbf{r}}_{\text{proj}}\hat{\mathbf{r}}_{\text{proj}}^\top\right)\mathbf{W}$
Melalui opsi --projected dan --normpreserve (yang mempertahankan magnitudo norma $\Vert\mathbf{W}\Vert$ pada hiperbola satuan), serta pencarian hiperparameter Optuna pada AIAnytime/ablate dan p-e-w/heretic v2.0 yang meminimalkan fungsi objektif gabungan $\mathcal{L}_{\text{opt}} = \text{Refusal_Rate} + \lambda \cdot D_{\text{KL}}(P_{\text{orig}}(y \mid x_{\text{benign}}) \parallel P_{\text{ablated}}(y \mid x_{\text{benign}}))$, tingkat penolakan model Qwen/Qwen3.5-4B berhasil ditekan dari 99/100 menjadi 4/100 dengan penyimpangan distribusi D_{KL} hanya sebesar 0,0509. Praktik terbaik ini juga dikonfirmasi oleh arsitektur huihui-ai/Huihui-Qwen3.8-27B-abliterated pada Oktober 2026 yang hanya mengablasi lapisan 18 hingga 51 sementara membiarkan lapisan 1–15 tidak tersentuh demi menjaga ketajaman kognitif model dasar, yang berjalan beriringan dengan lompatan akurasi penalaran pada DeepSeek-R1-0528 (684B) dan model distilasi kompaknya DeepSeek-R1-0528-Qwen3-8B.
Dari sisi keamanan siber, pengunduhan model *uncensored* dari repositori publik membawa risiko **Serangan Serialisasi Model (*Model Serialization Attack*)** apabila berkas bobot disimpan dalam format Python pickle (.bin, .pt, .pth, .pkl) yang mengeksekusi kode arbitrer saat fungsi torch.load() dipanggil. Sebagai contoh konkret dalam audit repositori Wan-AI/Wan2.1-T2V-1.3B, meskipun berkas utama diffusion_pytorch_model.safetensors (5,68 GB) murni berisi tensor numerik nir-eksekusi, berkas pendukung Wan2.1_VAE.pth (508 MB) dan models_t5_umt5-xxl-enc-bf16.pth (11,4 GB) masih menggunakan format pickle yang memuat instruksi impor torch._utils._rebuild_tensor_v2 dan collections.OrderedDict. Oleh karena itu, setiap pipa akuisisi model dalam arsitektur ini diwajibkan memprioritaskan format .safetensors dan .gguf, serta mengeksekusi pemindaian statis menggunakan **protectai/modelscan** sebelum memuat berkas non-Safetensors apa pun; pipeline otomatis hanya akan melanjutkan proses apabila modelscan mengembalikan kode keluar (*exit code*) 0 (bersih), dan langsung mengkarantina berkas apabila menerima kode keluar 1 (ditemukan muatan berbahaya), 2 (galat pemindaian), 3 (format tidak didukung), atau 4 (kesalahan argumen).

Cetak Biru Kode Otomasi Multi-Jalur (Self-Healing & Auto-Rolling Code Blueprints)

Agar agen pemrograman AI (seperti OpenClaw, Nous Hermes Agent, OpenHands, Aider, atau Pi) memiliki pemahaman struktural untuk merakit setiap kebutuhan secara *auto-rolling* tanpa menunggu intervensi manusia, sub-bagian berikut menyajikan cetak biru kode produksi yang mengikat seluruh tautan dari Kelompok 1, 2, dan 3 ke dalam mekanisme **Pemutus Sirkuit Multi-Jalur (*Multi-Tier Circuit Breaker*)**.

1. Standarisasi Brankas .env Master Multi-Jalur

Pengembangan dari templat .env pada berkas sumber ketiga dilakukan dengan mengubah setiap variabel tunggal menjadi pasangan rantai primer, sekunder, dan lokal. Sesuai aturan keamanan **Zero-Commit**, berkas .env ini wajib dimasukkan ke dalam .gitignore sejak inisialisasi

repositori pertama agar tidak pernah terunggah ke GitHub publik.

```
#
================================================================================
======
# MASTER .ENV BLUEPRINT — MULTI-PATH ROLLING & FAILOVER ARCHITECTURE
# Aturan Keamanan: WAJIB masuk .gitignore. Gunakan Virtual Visa
(FIN-SEC-01)
#
================================================================================
======

# --- 1. OMNICHANNEL & ADMIN CONTROL (AUTO-ROLLING 3 BOT TOKENS +
WAHA) ---
TOKEN_BOT_UTAMA=""
TOKEN_BOT_ACTIVITY=""
TOKEN_BOT_CADANGAN=""
ADMIN_USER_ID=""
GROUP_CHAT_ID="-100..."
TOPIC_ID_ACTIVITY=""
TOPIC_ID_ASSISTANCE=""
WAHA_LOCAL_URL="http://127.0.0.1:3000"
META_GRAPH_API_TOKEN=""

# --- 2. LAYER 1: EDGE NETWORK & PROXY HARVESTER ENDPOINTS ---
CF_WORKER_PROXY_URL="https://proxy-gateway.domain-anda.workers.dev"
CF_WORKER_VALID_KEY=""
LOCAL_PROXY_POOL_API="http://127.0.0.1:5010"
FORCE_DOH_DNS="https://1.1.1.1/dns-query"

# --- 3. LAYER 2: SELF-HOSTED GATEWAYS & SEMANTIC CACHE ---
REDIS_CONN_STRING="redis://default:password@endpoint.upstash.io:6379"
OMNIROUTE_BASE_URL="http://127.0.0.1:4000/v1"
NINEROUTER_BASE_URL="http://127.0.0.1:4001/v1"
NEW_API_HORIZON_URL="http://127.0.0.1:3001/v1"
PORTKEY_GATEWAY_URL="http://127.0.0.1:8787/v1"
SESSION_SECRET="gunakan_string_acak_kriptografis_minimal_64_karakter"
ROUTELLM_THRESHOLD="0.11593"

# --- 4. LAYER 3: TIER-1 CARDLESS FREE CLOUD LLM API KEYS (KELOMPOK
2A) ---
GEMINI_API_KEY=""
MISTRAL_API_KEY=""
GROQ_API_KEY=""
OPENROUTER_API_KEY=""
NVIDIA_NIM_API_KEY=""
CLOUDFLARE_ACCOUNT_ID=""
CLOUDFLARE_AI_TOKEN=""
SILICONFLOW_API_KEY=""
```

```
FIREWORKS_API_KEY=""
COHERE_API_KEY=""
HF_ACCESS_TOKEN="hf_..."
CHUTES_API_KEY=""
CEREBRAS_API_KEY=""

# --- 5. LAYER 4: TIER-2 LOCAL UNCENSORED & SOTA INFERENCE (KELOMPOK
1B/1D) ---
LLAMA_CPP_LOCAL_URL="http://127.0.0.1:8080/v1"
VLLM_LOCAL_URL="http://127.0.0.1:8000/v1"
OLLAMA_LOCAL_URL="http://127.0.0.1:11434/v1"
LOCAL_REASONING_MODEL="deepseek-ai/DeepSeek-R1-0528-Qwen3-8B"
LOCAL_UNCENSORED_MODEL="huihui-ai/Huihui-Qwen3.8-27B-abliterated-GGUF"
LOCAL_MOE_MODEL="huihui_ai/qwen3.6-abliterated:35b"

# --- 6. LAYER 5: MULTIMODAL, WEB SCRAPING & WHOLESALE FALLBACKS ---
SPIDER_CLOUD_API_KEY=""
CRAWL4AI_INTERNAL_URL="http://127.0.0.1:11235"
CIVITAI_API_KEY=""
FAL_KEY=""
NOVITA_API_KEY=""
WAVESPEED_API_KEY=""
REPLICATE_API_TOKEN=""
PIAPI_KEY=""

# --- 7. DATABASE, STORAGE & AUTONOMOUS TREASURY (PAYMENT ROLLING) ---
TURSO_DATABASE_URL="libsql://nama-db.turso.io"
TURSO_AUTH_TOKEN=""
SUPABASE_POSTGRES_DSN="postgresql://postgres:pass@db.supabase.co:5432/
postgres"
NEON_SQL_DSN="postgresql://user:pass@ep-cool.region.aws.neon.tech/neon
db"
BACKBLAZE_B2_KEY_ID=""
BACKBLAZE_B2_APP_KEY=""
PAKASIR_API_KEY=""
TRIPAY_API_KEY=""
DUITKU_API_KEY=""
CRYPTOBOT_API_TOKEN=""
```

2. Kode Python Rotasi Proksi & Perayap Web Multi-Jalur (Auto-Rolling Proxy & Scraper Engine)

Modul berikut menunjukkan cara kerja agen dalam menggabungkan kolam proksi lokal (jhao104/proxy_pool), cermin CDN (jsDelivr dan Fastly), umpan GitHub mentah (TheSpeedX, monosans, jetkai, roosterkid, BlackSnowDot), dan API publik (ProxyScrape, GeoNode). Lebih jauh lagi, fungsi akuisisi web merangkai tiga mesin perayap berbeda secara berurutan:

mencoba ekstraksi cepat menggunakan **spider_rs** terlebih dahulu, beralih secara otomatis ke **Scrapling StealthyFetcher** (untuk melewati Cloudflare Turnstile dan merelokasi elemen adaptif 12-faktor), dan menggunakan server internal **Crawl4AI** sebagai jalur terakhir.

```
import asyncio
import random
import httpx
from typing import List, Optional

class MultiPathProxyAndScraperEngine:
    """
    Mesin Rotasi Proksi & Akuisisi Web Multi-Jalur Otomatis.
    Tidak bergantung pada satu sumber proksi maupun satu pustaka web
scraper.
    """
    def __init__(self, local_pool_url: str = "http://127.0.0.1:5010"):
        self.local_pool_url = local_pool_url
        # Rantai Umpan Proksi Berjenjang: CDN Mirrors -> GitHub Raw ->
Public APIs
        self.proxy_feed_chains: List[str] = [

"https://cdn.jsdelivr.net/gh/monosans/proxy-list@main/proxies/http.txt
",

"https://fastly.jsdelivr.net/gh/jetkai/proxy-list@main/online-proxies/
txt/proxies-http.txt",

"https://fastly.jsdelivr.net/gh/roosterkid/openproxylist@main/HTTPS_RA
W.txt",

"https://raw.githubusercontent.com/TheSpeedX/PROXY-List/master/http.tx
t",

"https://raw.githubusercontent.com/BlackSnowDot/proxylist-update-every
-minute/main/http.txt",

"https://raw.githubusercontent.com/clarketm/proxy-list/master/proxy-li
st-raw.txt",

"https://raw.githubusercontent.com/ShiftyTR/Proxy-List/master/http.txt
",

"https://raw.githubusercontent.com/UptimerBot/proxy-list/main/proxies/
http.txt",

"https://raw.githubusercontent.com/RX4096/proxy-list/main/online/http.
txt",

"https://raw.githubusercontent.com/mmpx12/proxy-list/master/http.txt",
```

```
"https://api.proxyscrape.com/v2/?request=displayproxies&protocol=http&
timeout=5000&country=all",
            "https://www.proxy-list.download/api/v1/get?type=http",
            "https://api.openproxylist.xyz/http.txt"
        ]
        self.verified_proxies: List[str] = []

    async def get_working_proxy(self) -> Optional[str]:
        """Mengambil proksi dari daemon Redis lokal, otomatis failover
ke 13 umpan publik."""
        async with httpx.AsyncClient(timeout=4.0) as client:
            # Jalur 1: Coba daemon lokal jhao104/proxy_pool terlebih
dahulu
            try:
                resp = await client.get(f"{self.local_pool_url}/get/")
                if resp.status_code == 200 and "proxy" in resp.json():
                    return f"http://{resp.json()['proxy']}"
            except Exception:
                pass

            # Jalur 2: Jika kolam memori kosong, panen dari rantai CDN
& GitHub Raw secara acak
            if not self.verified_proxies:
                feeds = random.sample(self.proxy_feed_chains, k=min(4,
len(self.proxy_feed_chains)))
                for feed_url in feeds:
                    try:
                        r = await client.get(feed_url)
                        if r.status_code == 200:
                            lines = [
                                line.strip().split()[0]
                                for line in r.text.splitlines()
                                if ":" in line and not
line.startswith("#")
                            ]
                            self.verified_proxies.extend(lines[:150])
                    except Exception:
                        continue

            if self.verified_proxies:
                chosen = random.choice(self.verified_proxies)
                return f"http://{chosen}"
        return None

    async def report_dead_proxy(self, proxy_url: str) -> None:
        """Menghapus proksi mati dari memori dan melaporkannya ke
jhao104/proxy_pool."""
```

```
        raw_host = proxy_url.replace("http://",
"").replace("https://", "")
        if raw_host in self.verified_proxies:
            self.verified_proxies.remove(raw_host)
        try:
            async with httpx.AsyncClient(timeout=2.0) as client:
                await
client.get(f"{self.local_pool_url}/delete/?proxy={raw_host}")
        except Exception:
            pass

    async def scrape_with_rolling_fallback(self, target_url: str,
css_selector: str = "body") -> str:
        """
        Ekstraksi Web 3-Tingkat:
        Tier 1: Spider-RS (Rust HTTP/Smart Chrome Streaming)
        Tier 2: Scrapling StealthyFetcher (Cloudflare Turnstile Bypass
+ Adaptive 12-Factor Relocation)
        Tier 3: Crawl4AI Internal Docker Service (Isolated Network)
        """
        proxy = await self.get_working_proxy()

        # TIER 1: Spider-RS (Kecepatan Ekstrem Berbasis Rust)
        try:
            import spider_rs
            website = spider_rs.Website(target_url)
            website.crawl()
            pages = website.get_pages()
            if pages and len(pages[0].get_content()) > 100:
                return pages[0].get_content()
        except Exception:
            pass

        # TIER 2: Scrapling StealthyFetcher (Anti-Bot Bypass &
Adaptive Element Tracking)
        try:
            from scrapling.fetchers import StealthyFetcher
            StealthyFetcher.adaptive = True
            page = await StealthyFetcher.async_fetch(
                target_url,
                headless=True,
                network_idle=True,
                proxy=proxy
            )
            elements = page.css(css_selector, adaptive=True)
            if elements:
                return "\n".join([el.text for el in elements if
el.text])
```

```
        except Exception:
            if proxy:
                await self.report_dead_proxy(proxy)

        # TIER 3: Fallback ke Crawl4AI Internal Endpoint (Tanpa Ekspos
Publik untuk Cegah SSRF)
        async with httpx.AsyncClient(timeout=25.0) as client:
            resp = await client.post(
                "http://127.0.0.1:11235/crawl",
                json={"urls": [target_url], "priority": 10}
            )
            resp.raise_for_status()
            return resp.json().get("results", [{}])[0].get("markdown",
"")
```

3. Orkestrator Inferensi LLM, Keamanan ModelScan & Multimodal Auto-Rolling

Kode di bawah ini mengimplementasikan logika inti yang memastikan setiap permintaan kognitif maupun multimodal dari agen memiliki jalan keluar berlapis. Sebelum model lokal dimuat, fungsi verify_model_security() menjalankan protectai/modelscan secara terprogram untuk memblokir serangan serialisasi pickle. Selama waktu jalan (*runtime*), execute_llm_with_rolling() menghitung kompleksitas prompt dan merutekannya melintasi penyedia cloud gratis tanpa kartu kredit (Kelompok 2A), lalu beralih otomatis ke agregator lokal (OmniRoute dan 9Router) serta mesin GGUF tanpa sensor (llama.cpp dan Ollama dengan bobot huihui-ai atau DeepSeek-R1-0528-Qwen3-8B) apabila kuota eksternal habis.

```
import os
import subprocess
import time
from dataclasses import dataclass
from typing import Dict, List, Optional
import httpx

@dataclass
class ProviderEndpoint:
    name: str
    base_url: str
    api_key_env: str
    default_model: str
    tier: str  # "LOCAL_SOTA", "FREE_CLOUD", "AGGREGATOR", "WHOLESALE"
    cooldown_until: float = 0.0

class AutonomousRollingOrchestrator:
    """
    Orkestrator Inferensi Multi-Jalur dengan Circuit Breaker Otomatis,
    Perutean Ambang Batas Kompleksitas (RouteLLM tau=0.11593), dan
```

```
Audit ModelScan.
    """
    def __init__(self):
        self.tau_threshold = float(os.getenv("ROUTELLM_THRESHOLD",
"0.11593"))
        self.endpoints: List[ProviderEndpoint] = [
            # Jalur Cepat & Tanpa Sensor Lokal (Biaya Rp 0, Privasi
100%)
            ProviderEndpoint(
                name="LlamaCpp_Local_Abliterated",
                base_url=os.getenv("LLAMA_CPP_LOCAL_URL",
"http://127.0.0.1:8080/v1"),
                api_key_env="NONE",
                default_model=os.getenv("LOCAL_UNCENSORED_MODEL",
"huihui-ai/Huihui-Qwen3.8-27B-abliterated-GGUF"),
                tier="LOCAL_SOTA"
            ),
            ProviderEndpoint(
                name="Ollama_Local_Reasoning",
                base_url=os.getenv("OLLAMA_LOCAL_URL",
"http://127.0.0.1:11434/v1"),
                api_key_env="NONE",
                default_model="deepseek-ai/DeepSeek-R1-0528-Qwen3-8B",
                tier="LOCAL_SOTA"
            ),
            # Jalur Agregator Mandiri (150+ Penyedia Gratis &
Konverter CLI)
            ProviderEndpoint(
                name="OmniRoute_Gateway",
                base_url=os.getenv("OMNIROUTE_BASE_URL",
"http://127.0.0.1:4000/v1"),
                api_key_env="SESSION_SECRET",
                default_model="auto-free",
                tier="AGGREGATOR"
            ),
            ProviderEndpoint(
                name="9Router_Coding_Bridge",
                base_url=os.getenv("NINEROUTER_BASE_URL",
"http://127.0.0.1:4001/v1"),
                api_key_env="SESSION_SECRET",
                default_model="claude-code-free",
                tier="AGGREGATOR"
            ),
            # Jalur Cloud Gratis 100% Tanpa Kartu Visa (Kelompok 2A -
Terverifikasi 2026)
            ProviderEndpoint(
                name="Google_AI_Studio_Gemini",
```

```
base_url="https://generativelanguage.googleapis.com/v1beta/openai",
                api_key_env="GEMINI_API_KEY",
                default_model="gemini-3.5-flash",
                tier="FREE_CLOUD"
            ),
            ProviderEndpoint(
                name="Mistral_Experiment_1B_Pool",
                base_url="https://api.mistral.ai/v1",
                api_key_env="MISTRAL_API_KEY",
                default_model="mistral-large-latest",
                tier="FREE_CLOUD"
            ),
            ProviderEndpoint(
                name="Groq_LPU_Fast",
                base_url="https://api.groq.com/openai/v1",
                api_key_env="GROQ_API_KEY",
                default_model="qwen-qwq-32b",
                tier="FREE_CLOUD"
            ),
            ProviderEndpoint(
                name="OpenRouter_Dynamic_Free",
                base_url="https://openrouter.ai/api/v1",
                api_key_env="OPENROUTER_API_KEY",
                default_model="openrouter/free",
                tier="FREE_CLOUD"
            ),
            ProviderEndpoint(
                name="NVIDIA_NIM_91_Free",
                base_url="https://integrate.api.nvidia.com/v1",
                api_key_env="NVIDIA_NIM_API_KEY",
                default_model="deepseek-ai/deepseek-r1",
                tier="FREE_CLOUD"
            ),
            # Jalur Cadangan Grosir Berbasis Crypto USDT / Kredit
Pendaftaran (Kelompok 3A)
            ProviderEndpoint(
                name="SiliconFlow_Cloud",
                base_url="https://api.siliconflow.cn/v1",
                api_key_env="SILICONFLOW_API_KEY",
                default_model="deepseek-ai/DeepSeek-V3",
                tier="WHOLESALE"
            ),
            ProviderEndpoint(
                name="Novita_AI_USDT_Fallback",
                base_url="https://api.novita.ai/v3/openai",
                api_key_env="NOVITA_API_KEY",
                default_model="deepseek/deepseek-r1",
                tier="WHOLESALE"
```

```
            )
        ]

    @staticmethod
    def verify_model_security(model_path: str) -> bool:
        """
        Gerbang Zero-Trust: Memastikan berkas model adalah
.safetensors / .gguf
        atau lulus audit statis ProtectAI ModelScan (Exit Code 0).
        """
        if model_path.endswith((".safetensors", ".gguf")):
            return True
        res = subprocess.run(
            ["modelscan", "-p", model_path],
            capture_output=True,
            text=True,
            check=False
        )
        if res.returncode == 0:
            return True
        raise PermissionError(
            f"ModelScan memblokir pemuatan berkas '{model_path}' (Exit
Code {res.returncode}). "
            "Terdeteksi potensi instruksi berbahaya pada serialisasi
Pickle/H5!"
        )

    def _order_candidates(self, complexity_score: float,
require_uncensored: bool) -> List[ProviderEndpoint]:
        """Menyusun urutan prioritas penyedia berdasarkan skor
RouteLLM dan kebutuhan sensor."""
        now = time.time()
        active = [ep for ep in self.endpoints if ep.cooldown_until <=
now]
        if require_uncensored:
            # Prioritaskan model abliterated lokal dan agregator tanpa
filter
            priority = {"LOCAL_SOTA": 0, "AGGREGATOR": 1, "WHOLESALE":
2, "FREE_CLOUD": 3}
        elif complexity_score < self.tau_threshold:
            # Kueri ringan/menengah (< 0.11593) langsung diselesaikan
di lokal untuk hemat kuota
            priority = {"LOCAL_SOTA": 0, "FREE_CLOUD": 1,
"AGGREGATOR": 2, "WHOLESALE": 3}
        else:
            # Kueri kompleks (>= 0.11593) gunakan kuota gratis Cloud
Frontier -> Agregator -> Lokal
            priority = {"FREE_CLOUD": 0, "AGGREGATOR": 1,
```